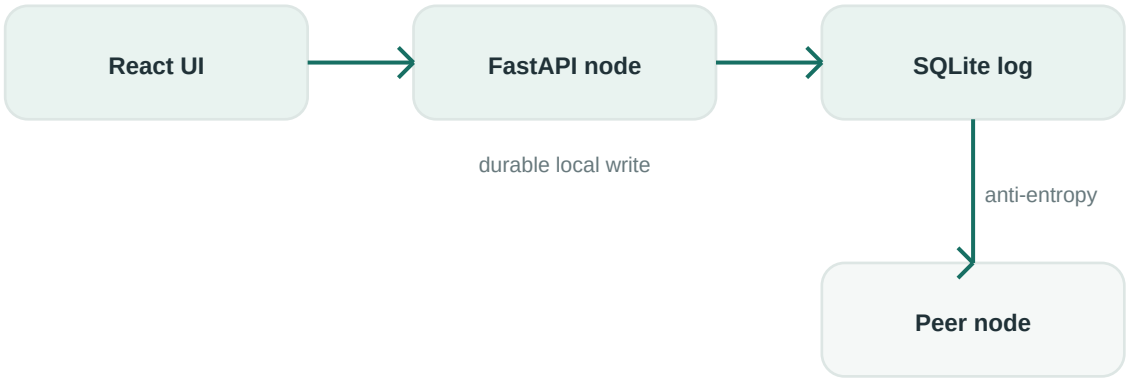
SyncMesh

How offline inventory changes move across independent stores

ONE NODE one private SQLite database	**ONE LOG** immutable ordered operations	**ONE GOAL** replicas converge when connected

The core idea

A shop must keep working when the network is unavailable. SyncMesh commits each change to that shop's own database first. A background exchange sends missing changes to peers when a connection returns. The shared state emerges from exchanging operations; there is no central database.



A single change, end to end

1. React dashboard
The operator creates or adjusts inventory at the selected location.

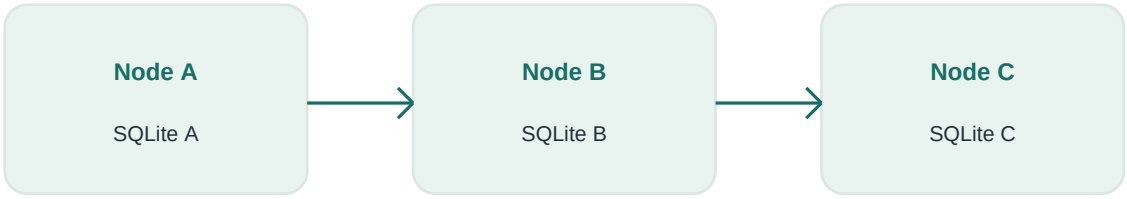
2. FastAPI node
The API validates the request and records an immutable operation.

3. SQLite transaction
The operation log and local inventory view are updated durably.

4. Peer exchange
When online, the node compares vectors and sends missing operations.

How replicas catch up

Three nodes keep their own histories and compare progress before transferring data.



Exchange only operations missing from each peer

Operation identity and vector clocks

Each operation carries an origin node ID and a sequence number, such as **shop-a:12**. A version vector such as **{shop-a: 12, shop-b: 4}** records the highest contiguous sequence applied from each origin. If sequence 12 arrives before 11, it waits in the pending queue until the gap closes. Duplicate operation IDs are ignored.

Conflict behavior

Change	Rule
Causally ordered field edit	An edit that knows the prior version supersedes it.
Concurrent field edits	Compare UTC timestamps; node ID and operation ID break ties.
Stock adjustment	Add signed deltas. Concurrent -2 and -3 changes total -5.
Delete	Keep a tombstone so an older copy cannot revive the item.

Run the demo

1. Install Docker Desktop and wait for its engine to start.
2. In the project folder run **docker compose up --build**.
3. Open **http://localhost:5173** and add an inventory item.
4. Select another node card and inspect its replica. Use **Sync now** to request an exchange.
5. Stop with Ctrl+C. Use **docker compose down -v** only when you intend to erase the demo databases.

Project map and learning path

Follow one request through the folders to understand where each responsibility belongs.

Folder	Purpose
src/syncmesh/protocol	Operation messages and version-vector math.
src/syncmesh/node/api	FastAPI request and WebSocket boundaries.
src/syncmesh/node/services	Peer synchronization use case.
src/syncmesh/node/storage	SQLite lifecycle, schema, operation log, pending queue.
frontend/src	React Router dashboard and replica views.
tests	Focused correctness checks for repository and vector behavior.

Try these experiments

Offline write: stop node-b, add an item on node-a, restart node-b, then compare item counts.
Stock merge: after syncing an item, adjust stock at two nodes before the next exchange. Additive deltas commute.
Inspect progress: open /sync/vector and /stats on ports 8001-8003 to read vector clocks and operation counts.

Prototype limitations

This project is an educational prototype. It does not provide authentication, encrypted transport, peer discovery, operation-log compaction, or production-grade conflict review. Keep demo endpoints on a trusted local network. The demo contains no measured performance claims.